PRD: SLR SLR Software (Single Lens Reflex Sightless Label Reader)

Team: #4 Colorado Dreaming (GEEN 1400, Fall 2026): Ethan Mitchell, Levi Nichols, Ore Porat, Coen Shields, Yitz Cohen **Doc owner:** Ore Porat **Status:** Draft v2 (adds GrocerEye model plan, measured hardware controls, UDP frame transport) **Audience:** Claude Code (implementer) and the team

---

0. Instructions for Claude Code

Read this whole document before writing code. Then:

1. Build in the milestone order in Section 12. Finish and demo each milestone before starting the next.
2. **Sim mode must always work.** The full product must run on a laptop alone (webcam + keyboard + laptop speakers) so the team can develop and test before the hardware is finished. Never break this.
3. Keep the guidance logic (detections in, audio cues out) as **pure, side-effect-free Python** with thorough unit tests. This is the heart of the product.
4. **Never install PyTorch, Ultralytics, OpenCV DNN models, or any ML runtime on the Raspberry Pi.** The Pi Zero 2 W has 512 MB of RAM. All inference runs on the laptop.
5. Everything runs **offline** at runtime. No cloud APIs, no internet required during use. (Downloading datasets and training on Colab beforehand is fine.)
6. Put every tunable number (thresholds, pins, ports, timings, calibration values) in YAML config, not in code.
7. When something in this PRD is ambiguous, pick the simplest option that satisfies the requirement, note the decision in `DECISIONS.md`, and keep going.

---

1. Problem

Visually impaired shoppers have trouble telling products apart on a store shelf and can grab the wrong item. Existing tools are mostly phone apps, which force the user to hold a phone in one hand while also managing a cane, a basket, or a cart. Asking store staff for help every time chips away at the user's independence.

2. Product summary

SLR SLR is a vintage SLR film camera, worn on a neck strap at chest height, retrofitted with a Raspberry Pi Zero 2 W and a Pi camera. The camera's original controls (shutter release, film advance lever, rewind knob, a volume dial, and a 3-position mode switch) become the user's controls. The Pi streams camera frames over WiFi to a laptop running a YOLO grocery detection model based on **GrocerEye** (Section 8.1). The laptop decides what to tell the user, and the Pi plays spoken and tonal audio cues through a headphone jack.

The user loads a shopping list, walks to the right aisle, and then the device:

- **Find mode:** guides them to an item on the shelf ("Pasta. Slightly left. Eye level. Within reach.")
- **Identify mode:** tells them what they are holding or pointing at ("This is coffee.")

What the model can and can't tell apart

GrocerEye recognizes **25 grocery categories** (pasta, rice, cereal, coffee, milk, soda, and so on), not specific brands. So v1 can tell pasta from rice or coffee from tea, which already prevents a lot of wrong grabs, but it cannot tell two pasta brands apart. Distinguishing specific look-alike products is a stretch goal (Section 10, Stage C).

Goals

- A blindfolded tester can find a named item category on a mock shelf, hands free, using only audio.
- The device never confidently names the wrong item (precision beats recall).
- Feels instant: audio feedback under half a second after the camera sees something.
- Learnable in under 10 minutes.

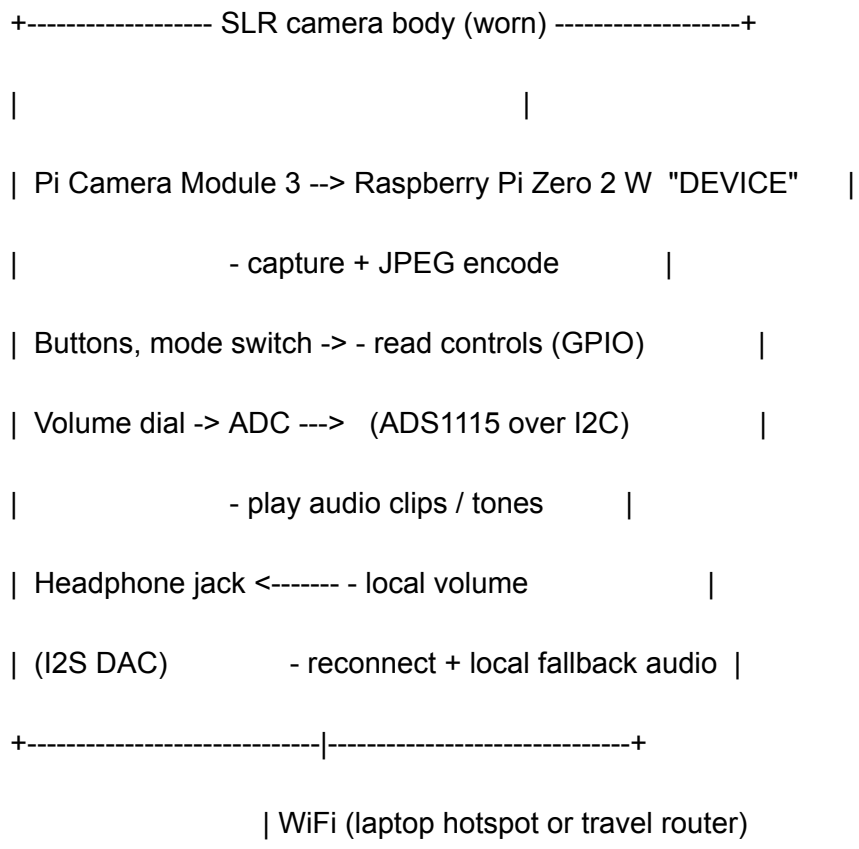
Non-goals (v1)

- Store-level navigation or finding the right aisle (the GPS concept was rejected at PDR).
- Reading text, prices, nutrition labels, or barcodes.
- Hazard detection (listed as a function in the workbook; out of scope for v1 software).
- Brand-level recognition (stretch goal only).
- Running the model on the Pi.
- A phone app or any cloud service.
- Braille output.

3. Users and test personas

Persona	**Description**	**What they need from the software**
Primary user	Blind or low vision shopper, cane or guide dog, carrying a basket	Hands-free, audio-only, reliable, never misleading, quick to learn
Helper	Friend or family member who sets up the shopping list	Easy list editing on the laptop
Team developer	One of us, building and debugging	Sim mode, live debug view, logs, fast retraining
Tester	Blindfolded teammate or TA during evaluation	Repeatable test protocol and measured results

4. System architecture



```
                    | UDP:        camera frames up, frame acks down
                    | WebSocket:  control events up, cues down
+---------------------------v-----------------------------+
|  Laptop  "SERVER"                                       |
|   - YOLO detector (GrocerEye-based model)               |
|   - Session state machine (shopping list, mode, current item)|
|   - Guidance engine (detections -> cues)  [pure logic]  |
|   - TTS clip renderer + cache (pushed to device)        |
|   - Debug view, logging, shopping list editor           |
+---------------------------------------------------------+
```

Design choice: thin device, smart server. The Pi only captures, plays audio, and reads controls. All logic lives on the laptop. Reasons: the Pi Zero 2 W is weak, logic on the laptop is much easier to debug and test, and sim mode becomes trivial (the "device" can just be a laptop process using the webcam).

Design choice: hybrid transport (UDP for frames, WebSocket for everything else). Camera frames go over UDP, as the PDR proposed: for live video, a lost frame should just be skipped, and UDP never stalls waiting to resend old data, which keeps lag low. Button presses and spoken cues go over a WebSocket (TCP) because they must never be lost: a dropped "Within reach" or a missed Enter press is a real bug for a blind user. Details in Section 7.

Tech stack

Area	**Choice**
Language	Python 3.11+ on both sides
Detector	Ultralytics YOLO nano (YOLO11n or the newest nano model Ultralytics recommends), trained on the GrocerEye dataset. Optional second backend: original GrocerEye YOLOv3 Darknet weights via OpenCV DNN

Area	Choice
Frame transport	Raw UDP sockets (`asyncio` datagram endpoints)
Control transport	`websockets` (asyncio)
Camera (Pi)	`picamera2` (install via apt, use a venv with `--system-site-packages`)
Camera (sim)	OpenCV `VideoCapture`
Controls (Pi)	`gpiozero` for buttons and mode switch; `adafruit-circuitpython-ads1x15` for the volume dial ADC
Controls (sim)	keyboard
Audio playback	`sounddevice` or `aplay`/ALSA; WAV clips; software gain for volume
TTS (server)	Piper (offline, natural voice) with `espeak-ng` as fallback
Config	YAML + pydantic validation
Tests	pytest
Labeling (our own photos)	Roboflow free tier or Label Studio, export YOLO format
Training	Google Colab GPU or laptop GPU

5. Hardware assumptions and notes

These affect the software. Items marked **measured** come from the hardware team.

- **Board:** Raspberry Pi Zero 2 W, Raspberry Pi OS Lite 64-bit (current release).
- **Space (measured):** there is plenty of room inside the body once the prism is removed.
- **Camera:** Pi Camera Module 3. The standard module has about 66° horizontal / 41° vertical FOV, which is narrow for a chest-mounted camera looking at a shelf. **Recommend the Camera Module 3 Wide (about 102° horizontal).** The Zero needs the smaller 22-pin camera ribbon cable.

- **Audio:** the Pi Zero 2 W has **no headphone jack and no analog audio out.** Recommended: an I2S DAC board with a 3.5 mm jack (PCM5102A type, `dtoverlay=hifiberry-dac`). Alternative: a USB audio dongle through a micro-USB OTG adapter. Software must work with any ALSA output device chosen in config. Stereo output is required for P1 panned tones.

- **Volume dial (measured):** a variable resistor reading **4.64 kΩ at minimum, 6.04 kΩ at the middle, 7.0 kΩ at maximum**, assumed to follow a logarithmic curve. **The Pi has no analog inputs**, so this needs an ADC. Recommended: an **ADS1115** (16-bit, I2C) reading a voltage divider made from the dial and a fixed resistor of about 5.6 kΩ to 3.3 V. That gives roughly 1.48 V to 1.82 V across the dial's travel, a small span that the ADS1115 resolves easily at its ±2.048 V gain setting (thousands of counts). Software calibrates from raw readings, so the exact fixed resistor value does not matter (FR-44).

- **Mode switch (measured):** a 3-position switch with two signal wires, **a** and **b**:

Position	**a**	**b**
OFF	0	0
ON	1	1
TOGGLE	0	1

  Each wire goes to a GPIO pin with an internal pull-up. Software treats "1" as "contact closed" (config has an `invert` flag in case the wiring is the other way). The state a=1, b=0 should never happen; treat it as a mid-movement glitch.

- **Buttons:** shutter release, film advance lever, rewind knob wired to GPIO as momentary contacts. Pins set in config.

- **Power:** battery pack. No battery level sensing in v1.

- **Network:** laptop runs a WiFi hotspot (or a small travel router) so the demo never depends on store or campus WiFi. Check that the hotspot allows device-to-laptop UDP (some hotspots isolate clients).

6. Functional requirements

Priority key: **P0** = required for the final demo. **P1** = should have. **P2** = nice to have.

6.1 Shopping list

ID	Pri	Requirement
FR-1	P0	The shopping list is an ordered list of class names stored on the laptop in `data/shopping_list.yaml`.
FR-2	P0	Only classes the loaded model knows can be added (the GrocerEye categories, plus any custom classes from Stage C). Unknown names are rejected with a clear error listing valid items.
FR-3	P0	CLI to view and edit the list: `slr list show`, `slr list add <item>`, `slr list remove <item>`, `slr list clear`.
FR-4	P1	A tiny local web page (served by the server) to edit the list. Must be fully screen-reader accessible: plain HTML form, real labels, no mouse-only controls.
FR-5	P0	Each item tracks a status: `pending` or `found`. Status persists across server restarts.
FR-6	P1	`slr list reset` marks all items pending again.

6.2 Find mode (core flow from PDR flow diagram)

ID	Pri	Requirement
FR-10	P0	User cycles through list items with controls. Each change announces the item: "Item 2 of 5: pasta."
FR-11	P0	Pressing Enter on an item starts a search: "Searching for pasta." Frame streaming starts.
FR-12	P0	**Item not in view:** if the target is not detected for `not_found_prompt_s` (default 4 s), say "Not seeing pasta. Turn slowly." Repeat at most every `not_found_repeat_s` (default 8 s).
FR-13	P0	**Item in view:** once the target is detected stably (Section 8.2), say "Pasta found," then give direction cues.
FR-14	P0	**Horizontal guidance:** "Left," "Slightly left," "Straight ahead," "Slightly right," "Right," based on the box center.
FR-15	P0	**Vertical guidance:** "High shelf," "Eye level," "Low shelf," based on the box center. Spoken once per acquisition and when it changes.
FR-16	P0	**Closeness:** when the target box is centered and large

ID	Pri	Requirement
		enough (calibrated threshold), say "Within reach. Pasta straight ahead."
FR-17	P0	Enter while within reach marks the item found: "Pasta checked off. Next: coffee." The next pending item becomes current and search stops.
FR-18	P0	Enter during a search cancels it: "Search stopped." Streaming stops.
FR-19	P0	**Lost target:** if a tracked target disappears for more than `lost_timeout_s` (default 1.5 s), say "Lost it," and go back to searching.
FR-20	P1	**Wrong item nearby:** if a different known item is detected confidently and centered while searching, say "That's rice, not pasta." Rate limited. Targets the "grabbing the wrong item" problem.
FR-21	P1	**Panned guidance tone:** while guiding, a short beep plays panned left/right toward the target, and the beep rate increases as the target gets more centered and closer (parking sensor style). Speech for state changes, tones for continuous guidance. Toggle in config.

ID	Pri	Requirement
FR-22	P2	"Getting closer" cue when box size is increasing over time.

6.3 Identify mode

ID	Pri	Requirement
FR-30	P0	Identify is triggered by moving the mode switch to TOGGLE, or by a long-press of the shutter release (≥ `long_press_s`, default 0.8 s). Both triggers are configurable.
FR-31	P0	Device captures a short burst (default 5 frames). Server picks the most confident detection nearest the center in each frame that arrives and takes a majority vote.
FR-32	P0	If the vote winner appears in at least 3 frames above `conf_identify`: "This is coffee." Otherwise: "Not sure. Hold it closer to the camera." If fewer than 3 frames arrive within 1.5 s (UDP loss), say "Not sure" rather than guessing.
FR-33	P1	If the identified item is on the shopping list and pending, add: "It's on your list."
FR-34	P0	After Identify, return to the previous state (resume search if one was active).

Note: the GrocerEye training images are close-up product shots, so Identify mode (item held near the camera) matches the model's strengths best. Expect it to work before Find mode does.

6.4 Controls

All physical inputs produce abstract events. The mapping lives in `config/controls.yaml` so it can change as wiring changes.

Event	Default physical control (Pi)	Sim key	Behavior
`next_item`	Film advance lever	→ or `n`	Next item in list, announce it
`prev_item`	Rewind knob	← or `p`	Previous item, announce it
`enter`	Shutter release (short press)	`Space`	Start search / check off / cancel (context dependent)
`identify`	Mode switch to TOGGLE, or shutter long press	`i`	Identify mode
`mode_on`	Mode switch to ON	`o`	Active: device responds to all controls. Says "Ready."
`mode_off`	Mode switch to OFF	`x`	Standby: stop any search, stop streaming, say "Paused." Only the mode switch and shutdown combo work in standby.
`volume_set(level)`	Volume dial (analog, via ADC)	`+` / `-`	Set volume 0 to 10, announce "Volume 6" once the dial settles

Event	Default physical control (Pi)	Sim key	Behavior
`help`	Hold film advance lever ≥ 2 s	`h`	Speak short instructions for every control
`status`	Hold rewind knob ≥ 2 s	`s`	"3 of 5 items found. Current item: milk."
`shutdown`	Hold shutter + advance together ≥ 5 s	`q`	"Shutting down," then safe OS shutdown (protects the SD card)

ID	Pri	Requirement
FR-40	P0	Buttons are debounced (default 50 ms). Long-press and short-press are distinguished.
FR-41	P0	Every control action gives audio confirmation (PDR flow: "headphone jack confirms item/volume").
FR-42	P0	**Mode switch decoding:** read wires a and b, map with the table in Section 5 (configurable). A new position must be stable for `switch_debounce_ms` (default 80 ms) before it counts. The invalid state (a=1, b=0) is ignored and logged. On boot, the device acts on the switch's current position.
FR-43	P1	If the TOGGLE position turns out to be latching (stays put) rather than spring-return,

ID	Pri	Requirement
		Identify runs once on entering TOGGLE, then the device waits; moving back to ON resumes normal use.
FR-44	P0	**Volume dial calibration:** `slr device calibrate-volume` walks the user through turning the dial to min, middle, and max and stores the raw ADC readings in `config/device.yaml`. Dial position is computed by piecewise interpolation through those three points (min = 0%, middle = 50%, max = 100%), which follows the measured nonlinear curve without assuming an exact formula. A `curve: log` option fits a logarithmic curve through the three points instead.
FR-45	P0	**Volume dial reading:** sample at ~20 Hz, median filter over 5 samples, quantize to 11 steps (0 to 10) with hysteresis of half a step so the level doesn't flicker at a boundary. Announce "Volume N" only after the dial has been still for 300 ms. Volume is applied on the Pi as a gain in dB (perceptually even steps), handled locally for zero latency, and reported to the server for logging.

ID	Pri	Requirement
FR-46	P1	If no ADC is detected at startup, fall back to two-button volume control (configurable pins) and log a warning.

6.5 System and device behavior

ID	Pri	Requirement
FR-50	P0	Device software starts automatically on boot (systemd service) and says "Starting up," then "Ready" once connected to the server.
FR-51	P0	Server address is in device config. P1: discover via mDNS (`_slrslr._tcp`).
FR-52	P0	If the WebSocket drops, the device plays a **locally stored** clip "Connection lost. Reconnecting," retries with backoff, and says "Connected" on success. These system clips ship with the device, not from the server.
FR-53	P0	On connect, the device syncs audio clips from the server (manifest with hashes; only download changed clips).
FR-54	P0	Frames are streamed **only** while searching or identifying. Idle and standby = camera not streaming (saves battery and bandwidth).

ID	Pri	Requirement
FR-55	P0	If the camera fails to start, say "Camera error" and keep controls and audio working.
FR-56	P0	If searching and no `frame_ack` has arrived for 3 s while the WebSocket is still up, say "Video connection weak" once (UDP path problem).
FR-57	P1	`help` gives a spoken tutorial on first boot (flag stored after first run).

6.6 Audio and speech

ID	Pri	Requirement
FR-60	P0	All phrases are **pre-rendered** on the laptop with Piper TTS into WAV clips (one clip per full phrase per item, for natural sound) and cached on the device. No TTS runs on the Pi.
FR-61	P0	A phrase catalog (`config/phrases.yaml`) defines every phrase template, for example `found: "{item} found."`. Clips are regenerated when classes or phrases change.
FR-62	P0	Each class has a spoken name separate from its class ID (`config/classes.yaml`:

ID	Pri	Requirement
		`tomato_sauce: "tomato sauce"`).
FR-63	P0	**Cue priority and interruption:** each cue has a priority (system > confirmation > guidance > info). A higher priority cue interrupts a lower one. Guidance cues never queue: a new one replaces any waiting one (only the freshest guidance matters).
FR-64	P0	**Rate limiting:** a direction cue is spoken only when the direction changes, or at most every `direction_repeat_s` (default 1.5 s) if unchanged.
FR-65	P1	Stereo panning support for tones (FR-21).
FR-66	P2	Speech rate setting.

6.7 Developer tooling

ID	Pri	Requirement
FR-70	P0	**Sim mode:** `slr sim` runs server + a simulated device on one laptop using webcam, keyboard, and laptop speakers. Same code paths (including UDP + WebSocket over localhost) except camera, controls, and audio backends.
FR-71	P0	**Debug view:** optional OpenCV window on the

ID	Pri	Requirement
		server showing the live frame, all boxes with class and confidence, the target highlighted, current state, last cue, latency, and UDP frame loss rate. Also useful for the poster and demo video.
FR-72	P0	**Session logs:** JSONL per session with timestamps for frames, detections, state changes, cues, latency, and dropped frames.
FR-73	P0	**Replay:** feed a recorded video or folder of frames through the server with a scripted control sequence and output the cue timeline. Used for regression tests.
FR-74	P1	Press `f` in the debug view to save the current frame to `data/captured/` for future training (collect failure cases).
FR-75	P0	`slr bench` measures end-to-end latency, FPS, and frame loss (Section 9).
FR-76	P1	`slr device test-controls` prints live readings of every button, the mode switch (a, b, decoded position), and the volume dial (raw ADC, position %, level). For the hardware team to verify wiring.

7. Communication protocol

Two channels between device (client) and server. Ports in config.

7.1 Frame channel: UDP (port 8766)

- **One frame = one datagram.** Device encodes JPEG and must keep each datagram ≤ `max_datagram_bytes` (default 60,000). If a frame is too big, re-encode at lower quality (start at quality 75, step down by 10, minimum 40), and keep using the lower quality for subsequent frames until frames are comfortably under the limit again. No chunking or reassembly.

- **Default resolution 640x480** (configurable). Note that the GrocerEye training images are only 224x224, so 416x312 or even 320x240 may work just as well and cuts frame size and loss. `slr bench` should compare.

- **Datagram layout:** fixed binary header then JPEG bytes:

Field	**Type**	**Notes**
magic	4 bytes `SLR1`	Reject anything else
frame_id	uint32	Increments per frame
session_id	uint32	Random per device boot, so restarts don't confuse frame ordering
capture_ts_ms	uint64	Device clock
purpose	uint8	0 = find, 1 = identify
burst_id	uint16	Identify burst number, 0 for find
width, height	uint16 each	
JPEG payload	rest	

- **Server keeps only the newest frame.** If a frame arrives older than the last one processed (lower `frame_id` in the same session), drop it. If several arrive while the detector is busy, process only the newest.

- **Frame ack (server to device, UDP):** after processing, the server sends a small datagram `{frame_id, infer_ms, server_ts}` back to the address the frame came from.

- **Pacing:** the device keeps at most **one frame in flight**: it sends a new frame when the previous ack arrives or after `ack_timeout_ms` (default 300 ms), whichever comes first, and never faster than `max_fps` (default 10). It always captures a fresh frame at send time and never sends a queued old one. Lost frames are simply skipped; nothing is resent.

7.2 Control channel: WebSocket over TCP (port 8765)

Reliable, ordered. JSON text messages with a `type` field.

Device to server

Type	Fields	Notes
`hello`	`device_id`, `session_id`, `udp_port`, `sw_version`, `clip_manifest_hash`	First message after connect; tells the server where to send UDP acks
`control`	`event`, `value` (optional), `ts`	Events from Section 6.4. `volume_set` includes the level.
`heartbeat`	`ts`, `frames_sent`, `acks_received`	Every 2 s; lets the server log UDP loss

Server to device

Type	Fields	Notes
`cue`	`cue_id`, `clip`, `priority`, `interrupt`	Play a cached clip
`tone`	`freq_hz`, `duration_ms`, `pan` (-1..1), `interval_ms`	Start/update guidance tone; `interval_ms: null` stops it
`stream`	`on`, `max_fps`, `purpose`, `burst` (optional N frames), `burst_id`	Start/stop UDP streaming; burst for Identify
`clips_sync`	list of `{name, sha256, url}`	Device fetches changed clips over HTTP from the server

8. Detection and guidance logic

8.1 Model: GrocerEye

GrocerEye is a YOLOv3 (Darknet) grocery detector trained on the **Freiburg Groceries** dataset converted to object detection (the "5K Groceries" annotations: 4,947 images, 25 classes, 224x224, Pascal VOC XML). It reports **84.59% mAP** and 70.10% average IoU on its test split. Its classes are categories (read the exact list from the dataset; expected: beans, cake, candy, cereal, chips, chocolate, coffee, corn, fish, flour, honey, jam, juice, milk, nuts, oil, pasta, rice, soda, spices, sugar, tea, tomato sauce, vinegar, water).

Two known limitations from the GrocerEye author: it **doesn't perform as well in natural settings or on video**, because the training images show products up close and large in frame, and packaging varies a lot. The images also come from German stores, so US packaging will look different.

How we use it: the GrocerEye repo uses old Darknet tooling and does not appear to publish its trained weights. So the plan is to **reproduce GrocerEye with modern Ultralytics YOLO on the same dataset** (Stage A), then fix the real-world gap with our own photos (Stage B). See Section 10. The detector interface supports two backends so the team can compare:

- `ultralytics` (default): our trained `.pt` / ONNX model.
- `darknet_opencv` (optional): original GrocerEye-style YOLOv3 `.cfg` + `.weights` loaded with `cv2.dnn.readNetFromDarknet`, if the team obtains them (by running GrocerEye's training notebook or contacting the author).

Use CUDA, Apple MPS, or CPU automatically. If CPU inference is too slow, export to ONNX or OpenVINO.

8.2 Stable detection

A target counts as **detected** when it appears with confidence ≥ `conf_find` (default 0.5, then set from TR-12) in at least `stable_k` of the last `stable_n` **processed** frames (default 3 of 5). Dropped UDP frames don't count as misses. This filters single-frame false positives. If multiple boxes of the target class exist, track the one with the highest confidence (P2: simple IoU tracking).

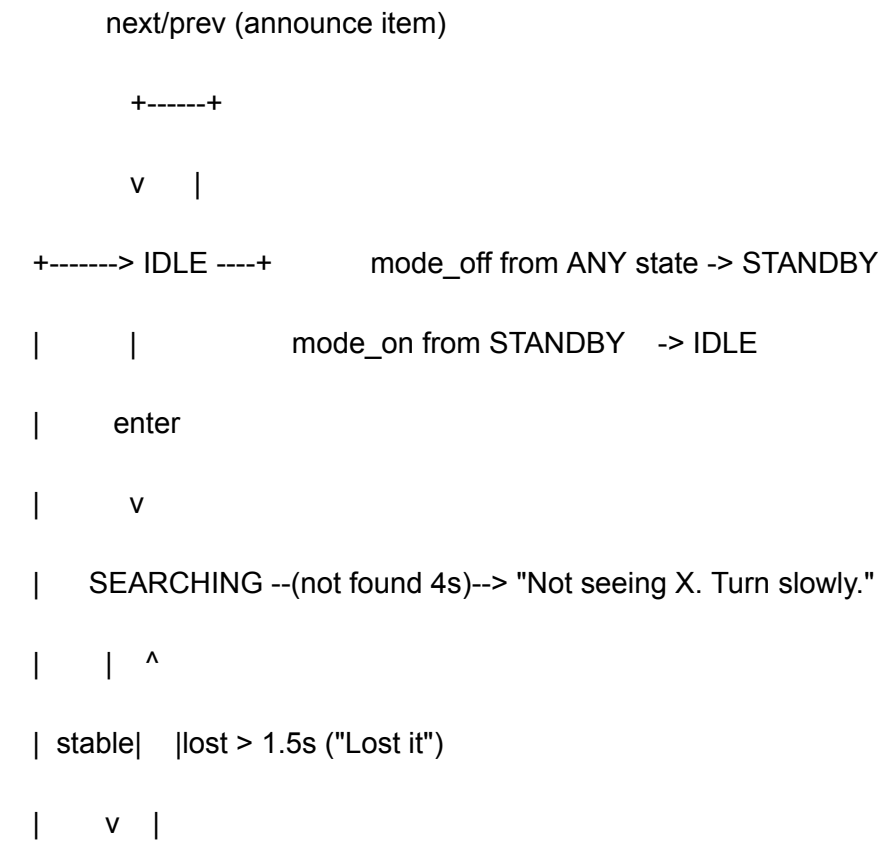
8.3 Direction math

Box center `(cx, cy)` and height `h`, all normalized to 0..1 of the frame. Apply `flip_horizontal` from config if the camera is mounted mirrored.

Cue	Rule (defaults, all configurable)
Straight ahead	`abs(cx - 0.5) < 0.08`
Slightly left / right	`0.08 ≤ abs(cx - 0.5) < 0.25`
Left / right	`abs(cx - 0.5) ≥ 0.25`
High shelf	`cy < 0.33`
Eye level	`0.33 ≤ cy ≤ 0.66`
Low shelf	`cy > 0.66`
Within reach	straight ahead **and** `h ≥ reach_h` (default 0.35, calibrate with real camera and mount angle) for ≥ 1 s

Vertical zones depend on how the camera is tilted on the chest. Add a `vertical_offset` config value and a calibration note in the README.

8.4 Session state machine



| GUIDING ---(centered + big for 1s)--> WITHIN_REACH

| |

+--- enter (cancel, from SEARCHING/GUIDING) | enter

| v

+---------- mark found, advance

identify from IDLE/SEARCHING/GUIDING/WITHIN_REACH -> IDENTIFYING -> back to previous state

`guidance.py` exposes pure functions: given the current state, config, recent detection history, and current time, return the new state plus a list of cues. No I/O. Time is passed in, never read from the clock, so tests are deterministic.

9. Non-functional requirements

These map to the user-centered criteria from the PDR workbook (Lag, Consistent, Identification, Ease of use, Environmental adaptability, Portable/battery).

ID	Criterion	Target	How we measure
NFR-1	Lag	Capture to start of audio cue **≤ 500 ms** (p90) on the demo laptop and hotspot	`slr bench` + session logs
NFR-2	Lag	Processed frame rate **≥ 5 FPS** while searching	`slr bench`
NFR-3	Lag	UDP frame loss **≤ 10%** at 1 to 5 m from the laptop	`slr bench`
NFR-4	Consistent	**Zero** wrong-item "found" or "This is" announcements in the acceptance test;	Acceptance test (Section 13)

ID	Criterion	Target	How we measure
		tune thresholds for precision over recall	
NFR-5	Identification	Stage A model: mAP50 on the Freiburg test split **≥ 0.85** (match or beat GrocerEye's 84.59%)	`training/evaluate.py`
NFR-6	Identification	Stage B model, on our own held-out photos of the demo categories: per-class precision **≥ 0.90**, recall **≥ 0.80**	`training/evaluate.py`
NFR-7	Identification	Blindfolded tester finds the named category on a mock shelf within 60 s in **≥ 8 of 10 trials**	Acceptance test
NFR-8	Ease of use	New user can operate all P0 controls after a ≤ 10 minute spoken tutorial	Test with someone outside the team
NFR-9	Environmental adaptability	Works under bright, dim, and mixed store-like lighting; no internet needed	Lighting variations in the acceptance test
NFR-10	Reliability	Recovers automatically from WiFi drop within 10 s of the network returning	Unplug/replug hotspot test
NFR-11	Startup	Device announces "Ready" within 60 s of power on	Stopwatch

ID	Criterion	Target	How we measure
NFR-12	Battery	Device does not stream frames when idle or in standby	Code review + log check
NFR-13	Device footprint	Device process uses < 150 MB RAM on the Pi Zero 2 W	`top` on device
NFR-14	Controls	Volume level does not flicker when the dial is untouched; mode switch never reports a false position	`slr device test-controls` for 2 minutes

10. Model training pipeline

Stage A (P0): Reproduce GrocerEye with modern YOLO

ID	Requirement
TR-1	`training/get_grocereye_data.py`: download the 5K Groceries object detection dataset (the Freiburg-based annotations GrocerEye used, linked from the GrocerEye README), and convert Pascal VOC XML to YOLO txt format. Read class names from the annotations; write `config/classes.yaml` and `data/grocereye/data.yaml`.
TR-2	Use the dataset's existing train/val split if provided; otherwise split 80/20 like GrocerEye, keeping a separate 10% test set. Record the split in the eval report.
TR-3	`training/train.py`: fine-tune YOLO nano from COCO weights. Images are 224x224, so train at `imgsz` 320 (configurable). About 100 epochs with early stopping, Ultralytics default augmentations plus brightness, blur, and scale variation (smaller objects help the

ID	Requirement
	"natural setting" gap). Runs locally or as a Colab notebook (`training/train_colab.ipynb`).
TR-4	`training/evaluate.py`: report mAP50, mAP50-95, per-class precision and recall, and a confusion matrix. Compare against GrocerEye's 84.59% mAP baseline. Output markdown to `reports/`.

Stage B (P0): Close the real-world gap with our own photos

GrocerEye's own author says it struggles outside close-up photos, so this stage is needed for a reliable demo.

ID	Requirement
TR-10	Pick **8 to 12 GrocerEye categories** for the demo that are cheap, non-perishable, and easy to bring (for example pasta, rice, cereal, coffee, tea, beans, soda, chips, jam, tomato sauce). Include categories that look alike in packaging (pasta vs rice bags, coffee vs tea boxes, juice vs soda bottles).
TR-11	`training/capture.py`: capture images from the laptop webcam **or the Pi camera through the SLR** (domain match matters) at a keypress or interval, saving to `data/raw/<session>/`.
TR-12	Collect 50 to 100 images per demo category: shelf distance (0.5 to 2 m) and close-up, several brands per category (US packaging), varied lighting, crowded backgrounds with other products present. Add background-only images (about 10% of the set) to reduce false positives. Label in Roboflow or Label Studio, export YOLO format. Split by capture session, not randomly, so near-duplicates don't leak into the test set.

ID	Requirement
TR-13	Fine-tune the Stage A model on Freiburg data **plus** our photos together (so it doesn't forget the other categories).
TR-14	`training/pick_threshold.py`: sweep confidence thresholds on our held-out photos and recommend the lowest one meeting per-class precision ≥ 0.90; write it to server config.
TR-15	Trained weights are versioned in `models/` with the eval report alongside. The server config points at one weights file.
TR-16	(P1) Failure frames saved via FR-74 get added to the dataset for the next training round.

Stage C (P2 stretch): Tell specific look-alike products apart

ID	Requirement
TR-20	Add product-specific classes for 2 or 3 confusable pairs (for example two cereal brands), 100+ images each, trained alongside the categories. Spoken names use the product name ("Honey Nut Cheerios"). This restores the PDR's "tell similar items apart" story at the brand level.

11. Repository layout

```
slr-slr/
  PRD.md
  README.md              # setup for laptop, Pi, and sim; wiring + calibration; demo checklist
  DECISIONS.md           # decisions made during implementation
  pyproject.toml         # extras: [server], [device], [training], [dev]
```

```
config/
  server.yaml            # model path + backend, thresholds, ports, TTS voice
  device.yaml            # server host, ports, pins, ADC + volume calibration, audio device, camera
  classes.yaml           # class_id -> spoken name (generated from dataset, then hand-edited)
  phrases.yaml           # phrase templates
  controls.yaml          # physical input -> event mapping, mode switch truth table
slr/
  common/
    protocol.py          # UDP header pack/unpack, WebSocket message dataclasses
    config.py            # pydantic config models + loader
  server/
    main.py              # orchestration
    frame_receiver.py    # UDP listener, newest-frame-wins, acks
    control_server.py    # WebSocket server
    detector.py          # backends: ultralytics, darknet_opencv -> list[Detection]
    guidance.py          # PURE: state + detections + time -> state + cues
    session.py           # shopping list, item status, persistence
    clips.py             # Piper TTS rendering + manifest
    debug_view.py
    list_web.py          # P1 accessible list editor
  device/
```

```
    main.py
    control_client.py     # WebSocket client, reconnect
    frame_sender.py       # UDP sender, one-in-flight pacing, adaptive JPEG quality
    camera.py             # picamera2 backend + OpenCV backend
    controls.py           # gpiozero buttons + mode switch decoder; keyboard backend
    volume_dial.py        # ADS1115 reader, filtering, calibration, hysteresis
    audio.py              # clip playback, priority/interrupt, volume, tones
  cli.py                  # `slr`: sim, server, device, list, bench, replay, device calibrate-volume, device
test-controls
 training/
  get_grocereye_data.py
  capture.py
  train.py
  train_colab.ipynb
  evaluate.py
  pick_threshold.py
 device_assets/
  system_clips/           # local "connection lost", "starting up", "paused", etc.
 deploy/
  install_pi.sh           # apt deps, venv, I2C + I2S enable, systemd install
  slr-device.service
 tests/
```

```
  test_guidance.py        # most important test file
  test_session.py
  test_protocol.py        # UDP header round trip, size limits
  test_frame_receiver.py  # out-of-order and stale frame dropping
  test_mode_switch.py     # truth table, debounce, invalid state
  test_volume_dial.py     # calibration interpolation, hysteresis, settle timing
  test_audio_priority.py
  test_replay.py
data/                   # gitignored except shopping_list.yaml example
models/                 # gitignored weights, tracked eval reports
```

12. Milestones

Each milestone ends with something the team can run and see.

M0: Scaffold Repo layout, `pyproject.toml`, config loading with validation, protocol code with tests, `slr` CLI skeleton, README stub. Done when `pytest` passes and `slr --help` works.

M1: End-to-end in sim with a stock COCO model Detector wrapper, guidance engine + state machine with full unit tests, Piper clip rendering, sim device (webcam, keyboard, speakers), UDP frame channel + WebSocket control channel over localhost, debug view, session logs. Done when, on a laptop alone, you can put "bottle" on the list, press Space, and get guided to a water bottle by voice. (COCO is used here only so the pipeline can be built before training finishes.)

M2: GrocerEye model (Stage A) Dataset download + VOC to YOLO conversion, train/evaluate scripts, Colab notebook. Done when the eval report exists, the server runs with the new model, and Identify mode correctly names close-up products in sim.

M3: Real device on the Pi Zero 2 W picamera2 backend, gpiozero buttons, mode switch decoder, ADS1115 volume dial with calibration, ALSA audio backend, local system clips, clip sync, reconnect logic, `test-controls` tool, systemd service, `install_pi.sh`. Done when the Pi boots, says "Ready," and the M2 demo works through the SLR with headphones.

M4: Real-world model (Stage B) Capture tool, our photos labeled, combined fine-tune, threshold picking. Done when NFR-6 is met (or the report shows exactly which categories fall short).

M5: Polish (P1 features) Wrong-item warnings, panned tones, "It's on your list," first-run tutorial, accessible list web page, mDNS, failure frame capture, replay regression tests.

M6: Acceptance and demo hardening `slr bench`, acceptance test script and results template, demo checklist in README (charge battery, start hotspot, start server, check volume, reset list, mode switch to ON).

13. Acceptance test protocol (for CDR / final)

1. Mock shelf: 3 shelf heights, all demo categories placed among some distractor products, look-alike categories placed next to each other.
2. Tester is blindfolded, wears the device, stands 1.5 m from the shelf facing roughly toward it.
3. For each of 10 trials, a teammate puts a random category on the list and the tester presses Enter. Stopwatch starts.
4. Trial succeeds if the tester's hand touches a correct item within 60 s. Record time, success, and any wrong-item announcement.
5. Also run 10 Identify trials: hand the tester a random item, record whether the announcement was correct, "not sure," or wrong.
6. Repeat a subset under dim lighting.
7. Pull latency and frame loss stats from session logs.
8. Record everything in `reports/acceptance_<date>.md`.

Pass: NFR-1, NFR-4, NFR-7 met.

14. Risks

Risk	Impact	Mitigation
GrocerEye-style model weak in real stores (author's own note)	Items missed at shelf distance	Stage B photos through our own camera, scale augmentation, Identify mode as the reliable fallback
Category-level model can't tell brands apart	Weaker story for "similar items"	Be upfront at CDR; Stage C stretch for 2 to 3 look-alike pairs

Risk	Impact	Mitigation
Wrong item announced	User grabs wrong thing; core purpose fails	Precision-first thresholds, stable-detection filter, "not sure" responses
UDP frame loss on WiFi	Choppy detection	One-in-flight pacing, adaptive JPEG quality, lower resolution option, loss shown in debug view and bench
Hotspot blocks UDP between clients	No video at all	Test early; FR-56 warning; travel router as backup
Pi has no analog input	Volume dial unreadable	ADS1115 ADC; two-button fallback (FR-46)
Small voltage span from volume dial (≈0.34 V)	Noisy volume	16-bit ADC at ±2.048 V gain, median filter, hysteresis
Camera FOV too narrow	Item not in view, lots of turning	Camera Module 3 Wide; "turn slowly" prompts; vertical offset calibration
No audio out on Pi Zero 2 W	No headphone output	I2S DAC with 3.5 mm jack; audio backend configurable
Hardware not ready in time	Software can't be tested	Sim mode is first-class; M1 and M2 need no Pi at all
SD card corruption from yanking power	Device won't boot at demo	Safe shutdown combo; standby via mode switch; README demo checklist
Fitting parts in the body	Reduced, prism removed and space is plentiful (measured)	No software impact

15. Open questions for the team

1. Is the mode switch's TOGGLE position spring-return (pops back) or latching (stays)? This decides FR-43. What did the switch originally do on the camera?
2. Does the volume dial's resistance change smoothly (min → middle → max) in one direction across its full travel?
3. Standard or Wide Camera Module 3? (Recommend Wide.)
4. Audio hardware: I2S DAC board or USB dongle?

5. ADC: OK to add an ADS1115 board? It shares the I2C pins, so check it doesn't conflict with the I2S DAC pins (it shouldn't).
6. Final list of 8 to 12 demo categories.
7. Laptop specs (GPU? Mac?) and which laptop runs the hotspot at the demo.
8. Where on the chest and at what tilt will the camera sit? (Needed for vertical zone calibration.)

16. Glossary

- **YOLO:** "You Only Look Once," a fast object detection model family. We train with the Ultralytics implementation.
- **GrocerEye:** an open-source YOLOv3 grocery detector trained on the Freiburg Groceries dataset; our starting point.
- **mAP50:** standard detection accuracy score; higher is better.
- **Precision:** of the times the model says "pasta," how often it really is pasta. Most important metric for us.
- **Recall:** of all the times pasta is in view, how often the model notices.
- **UDP:** a network protocol that sends packets with no delivery guarantee. Good for live video (skip lost frames), bad for button presses.
- **WebSocket / TCP:** a reliable, ordered connection. Used for controls and spoken cues.
- **ADC:** analog-to-digital converter. Lets the Pi read the volume dial's resistance.
- **I2S DAC:** a small add-on board that gives the Pi real audio output.